Applied Machine Learning

Learning goals

- Basics of mlr3:
  Train, Predict, Evaluate
- Resampling in mlr3
- Benchmarking in mlr3

APPLIED MACHINE LEARNING

Introduction to mlr3
- Motivation and R6
- Data and Tasks
- Dictionaries
- Learner
- Prediction and Evaluation

Resampling with mlr3

Benchmarking with mlr3

Help and Summary

SO YOU WANT TO DO ML IN R

- R gives you access to many machine learning methods
- ...but without a unified interface
- things like performance evaluation are cumbersome

Example:

```
# Specify what we want to model in a formula: target ~ features
svm_model = e1071::svm(Species ~ ., data = iris)
```

vs.

```
# Pass the features as a matrix and the target as a vector
xgb_model = xgboost::xgboost(data = as.matrix(iris[1:4]),
  label = iris$Species, nrounds = 10)
```

SO YOU WANT TO DO ML IN R

```r
library("mlr3")
```

Ingredients:

- Data / Task
- Learning Algorithms
- Performance Evaluation
- Performance Comparison

R6 – ALL YOU NEED TO KNOW

`mlr3` uses the *R6* class system which facilitates OOP by allowing the creation of custom objects with methods and properties (it may look unusual if you see it the first time).

- *Objects* are created using `<Class>$new()`.

```
task = TaskClassif$new(id = "iris", backend = iris, target = "Species")
```

- Alternatively, the function `as_task_classif` can be used (or `as_task_regr` to construct a `TaskRegr` object for regression tasks). By default, the name of the object passed to `x` is used as `id`:

```
task = as_task_classif(x = iris, target = "Species")
```

R6 – ALL YOU NEED TO KNOW

`mlr3` uses the *R6* class system which facilitates OOP by allowing the creation of custom objects with methods and properties (it may look unusual if you see it the first time).

- *Objects* are created using `<Class>$new()`.

```
task = TaskClassif$new(id = "iris", backend = iris, target = "Species")
```

- Alternatively, the function `as_task_classif` can be used (or `as_task_regr` to construct a `TaskRegr` object for regression tasks). By default, the name of the object passed to `x` is used as `id`:

```
task = as_task_classif(x = iris, target = "Species")
```

- Objects have *fields* that contain information about the object.

```
task$nrow
#> [1] 150
```

R6 – ALL YOU NEED TO KNOW

`mlr3` uses the *R6* class system which facilitates OOP by allowing the creation of custom objects with methods and properties (it may look unusual if you see it the first time).

- *Objects* are created using `<Class>$new()`.

```
task = TaskClassif$new(id = "iris", backend = iris, target = "Species")
```

- Alternatively, the function `as_task_classif` can be used (or `as_task_regr` to construct a `TaskRegr` object for regression tasks). By default, the name of the object passed to `x` is used as `id`:

```
task = as_task_classif(x = iris, target = "Species")
```

- Objects have *fields* that contain information about the object.

```
task$nrow
#> [1] 150
```

- Objects have *methods* that are called like functions:

```
task$filter(rows = 1:10)
```

R6 – ALL YOU NEED TO KNOW

`mlr3` uses the *R6* class system which facilitates OOP by allowing the creation of custom objects with methods and properties (it may look unusual if you see it the first time).

- *Objects* are created using `<Class>$new()`.

```
task = TaskClassif$new(id = "iris", backend = iris, target = "Species")
```

- Alternatively, the function `as_task_classif` can be used (or `as_task_regr` to construct a `TaskRegr` object for regression tasks). By default, the name of the object passed to `x` is used as `id`:

```
task = as_task_classif(x = iris, target = "Species")
```

- Objects have *fields* that contain information about the object.

```
task$nrow
#> [1] 150
```

- Objects have *methods* that are called like functions:

```
task$filter(rows = 1:10)
```

- Methods may change ("mutate") the object (reference semantics)!

```
task$nrow
#> [1] 10
```

R6 AND ACTIVE BINDINGS

Some fields of R6-objects may be *"Active Bindings"*. Internally they are realized as functions that are called whenever the value is set or retrieved.

- Active bindings for read-only fields

```
task$nrow = 11
#> Error in assert_ro_binding(rhs):  Field/Binding is
read-only
```

R6 AND ACTIVE BINDINGS

Some fields of R6-objects may be *"Active Bindings"*. Internally they are realized as functions that are called whenever the value is set or retrieved.

- Active bindings for read-only fields

```
task$nrow = 11
#> Error in assert_ro_binding(rhs):  Field/Binding is
read-only
```

- Active bindings for argument checking

```
task$properties = NULL
#> Error in assert_set(rhs, .var.name = "properties"):
Assertion on 'properties' failed:  Must be of type
'character', not 'NULL'.
task$properties = c("property1", "property2")  # works
```

MLR3 PHILOSOPHY

- Overcome limitations of S3 with the help of **R6**
  - Truly object-oriented: data and methods live in the same object
  - Make use of inheritance
  - Reference semantics

MLR3 PHILOSOPHY

- Overcome limitations of S3 with the help of **R6**
  - Truly object-oriented: data and methods live in the same object
  - Make use of inheritance
  - Reference semantics
- Embrace **data.table**, both for arguments and internally
  - Fast operations for tabular data
  - List columns to arrange complex objects in tabular structure

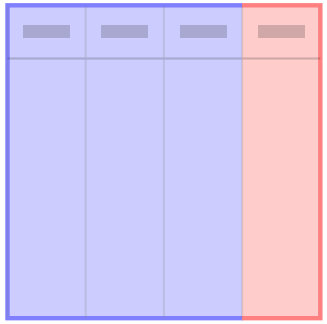
MLR3 PHILOSOPHY

- Overcome limitations of S3 with the help of **R6**
  - Truly object-oriented: data and methods live in the same object
  - Make use of inheritance
  - Reference semantics
- Embrace **data.table**, both for arguments and internally
  - Fast operations for tabular data
  - List columns to arrange complex objects in tabular structure
- Be **light on dependencies**:
  - `R6`, `data.table`, `lgr`, `uuid`, `mlbench`, `digest`
  - Plus some of our own packages (`backports`, `checkmate`, `...`)

DATA

- Tabular data
- Features
- Target / outcome to predict
  - discrete for classification
  - continuous for regression

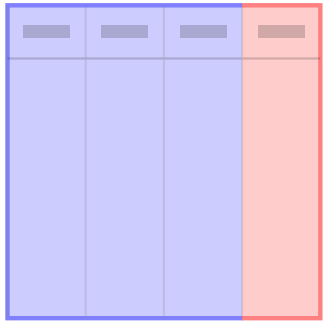
  $\Rightarrow$ target determines the machine learning "Task"



DATA

- Tabular data
- Features
- Target / outcome to predict
  - discrete for classification
  - continuous for regression

  ⇒ target determines the machine learning "Task"

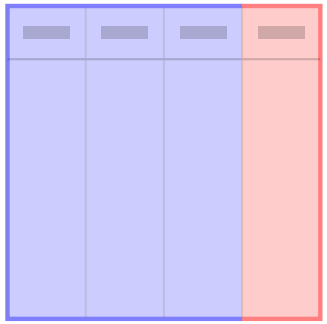


```
print(iris)  # included in R

#>   Sepal.Length Sepal.Width Petal.Length Petal.Width Species
#> 1          5.1         3.5          1.4         0.2  setosa
#> 2          4.9         3.0          1.4         0.2  setosa
#> ...
```

DATA

- Tabular data
- Features
- Target / outcome to predict
  - discrete for classification
  - continuous for regression

  ⇒ target determines the machine learning "Task"



```r
print(iris)  # included in R

#>   Sepal.Length Sepal.Width Petal.Length Petal.Width Species
#> 1          5.1         3.5          1.4         0.2  setosa
#> 2          4.9         3.0          1.4         0.2  setosa
#> ...
```

Task ID ↘ data ↓ target name ↙

```r
task = TaskClassif$new("iris", iris, "Species")
task = as_task_classif(x = iris, target = "Species", id = "iris")
```

DATA

```r
task = as_task_classif(x = iris, target = "Species")
```

```r
print(task)

# <TaskClassif:iris> (150 x 5)
# * Target: Species
# * Properties: multiclass
# * Features (4):
#   - dbl (4): Petal.Length, Petal.Width, Sepal.Length,
#     Sepal.Width
```

```r
task$ncol
task$nrow
task$feature_names
task$target_names
```

```r
task$head(n = )
task$truth(row_ids = )
task$data(rows = ,
          cols = )
```

```r
task$select(cols = )
task$filter(rows = )
task$cbind(data = )
task$rbind(data = )
```

DICTIONARIES

- `mlr3` uses `R6` classes to create dictionaries that store key-value pairs, i.e., associate keys (unique identifiers) with values (R6 objects).
- Dictionaries are easily extendable and allow adding and removing key-value pairs, e.g., add-on packages such as `mlr3learners` populate dictionaries with additional key-value pairs.

DICTIONARIES

- `mlr3` uses `R6` classes to create dictionaries that store key-value pairs, i.e., associate keys (unique identifiers) with values (R6 objects).
- Dictionaries are easily extendable and allow adding and removing key-value pairs, e.g., add-on packages such as `mlr3learners` populate dictionaries with additional key-value pairs.
- `mlr3` offers *Short Form* functions to get objects from a `Dictionary`:

Object	Dictionary	Short Form
`Task`	`mlr_tasks`	`tsk()`
`Learner`	`mlr_learners`	`lrn()`
`Measure`	`mlr_measures`	`msr()`
`Resampling`	`mlr_resamplings`	`rsmp()`

DICTIONARIES

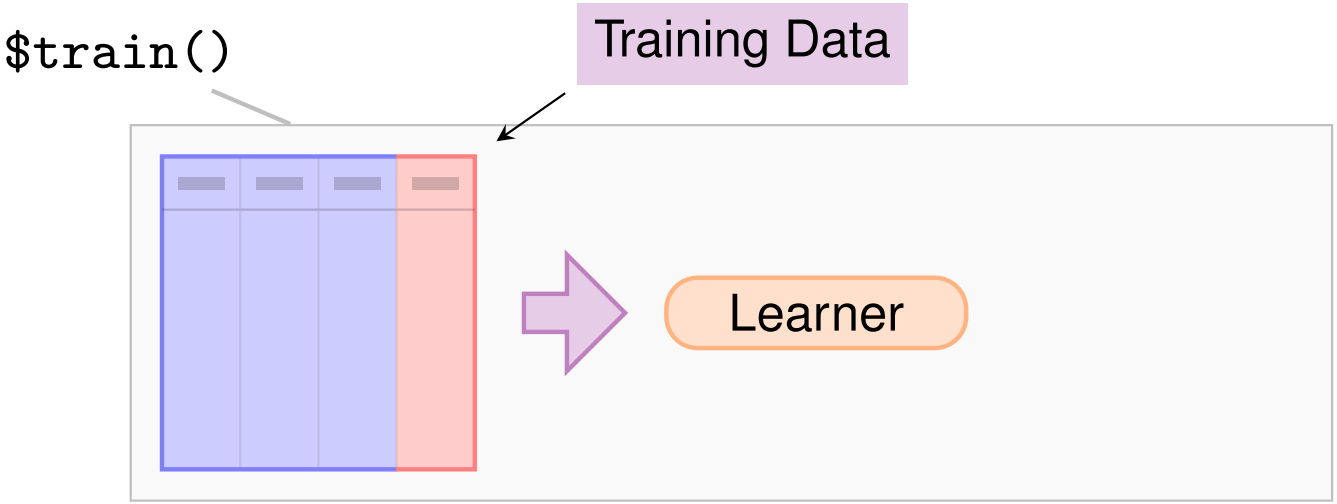
```r
# list items
tsk()

#> <DictionaryTask> with 11 stored values
#> Keys: boston_housing, breast_cancer, german_credit, iris,
#>   mtcars, penguins, pima, sonar, spam, wine, zoo

# retrieve object
tsk("iris")

#> <TaskClassif:iris> (150 x 5): Iris Flowers
#> * Target: Species
#> * Properties: multiclass
#> * Features (4):
#>   - dbl (4): Petal.Length, Petal.Width, Sepal.Length,
#>     Sepal.Width
```

SHORT FORMS AND DICTIONARIES

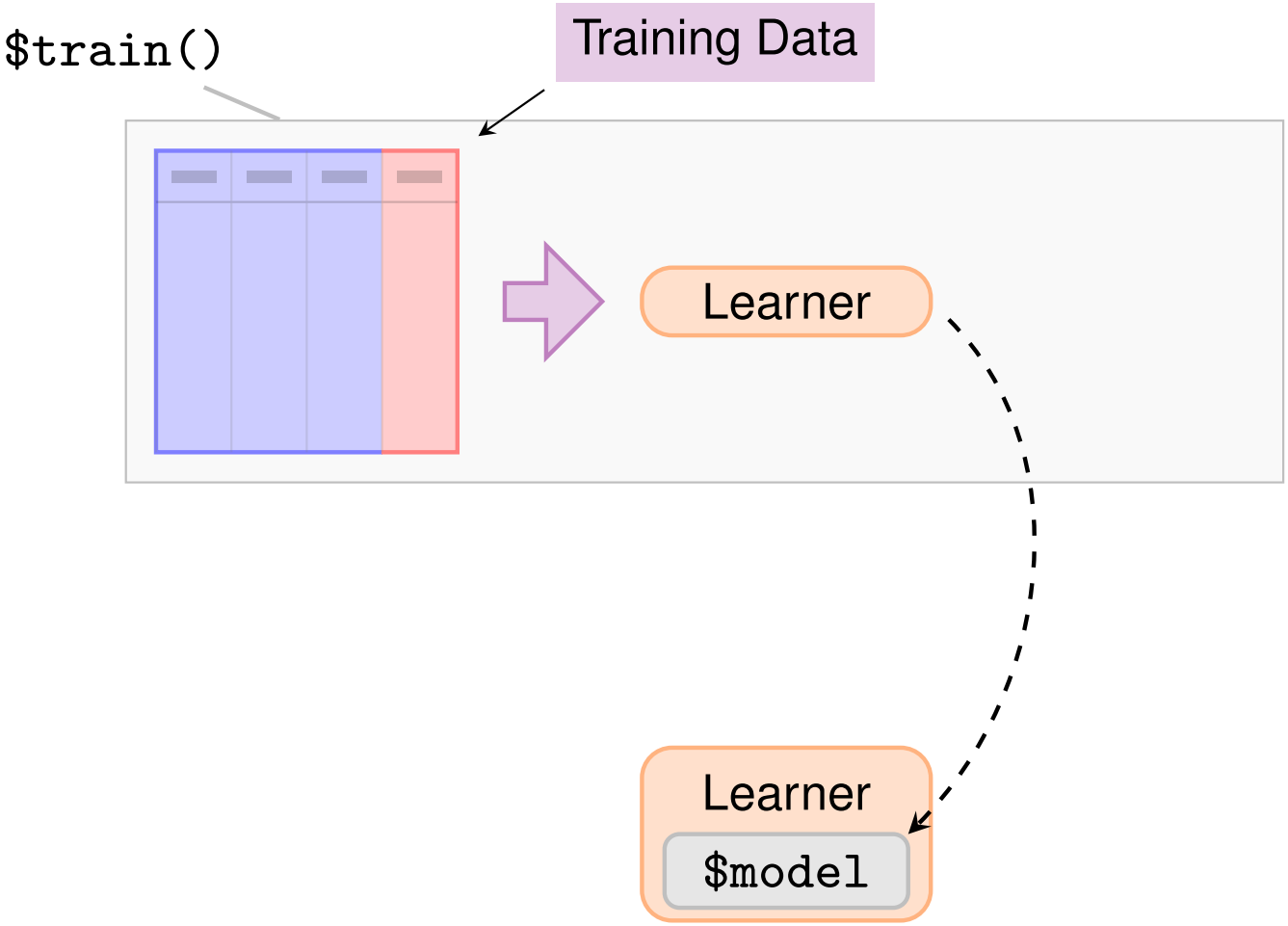
`as.data.table(<DICTIONARY>)` creates a `data.table` with metadata about objects in dictionaries:

```
as.data.table(mlr_learners)[1:5, c("key", "packages", "predict_types")]

# Key: <key>
#                    key   packages predict_types
#                 <char>     <list>        <list>
# 1:        classif.debug       mlr3 response,prob
# 2: classif.featureless       mlr3 response,prob
# 3:        classif.rpart mlr3,rpart response,prob
# 4:           regr.debug       mlr3   response,se
# 5:     regr.featureless mlr3,stats   response,se

library(mlr3learners) # mlr_learners dictionary gets populated
as.data.table(mlr_learners)[1:5, c("key", "packages", "predict_types")]

# Key: <key>
#                    key                  packages predict_types
#                 <char>                    <list>        <list>
# 1:   classif.cv_glmnet mlr3,mlr3learners,glmnet response,prob
# 2:        classif.debug                     mlr3 response,prob
# 3: classif.featureless                     mlr3 response,prob
# 4:       classif.glmnet mlr3,mlr3learners,glmnet response,prob
# 5:         classif.kknn   mlr3,mlr3learners,kknn response,prob
```

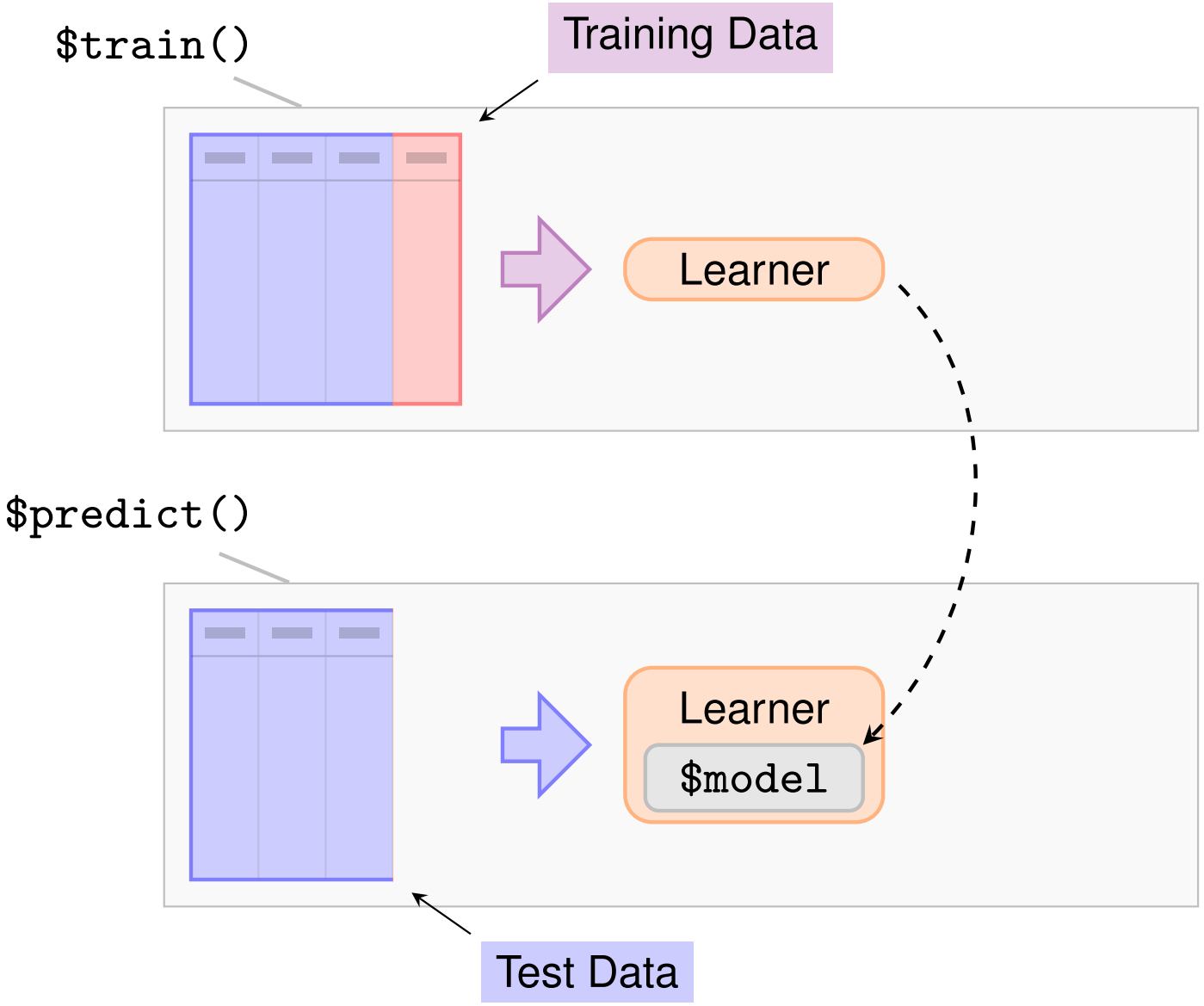
LEARNING ALGORITHMS



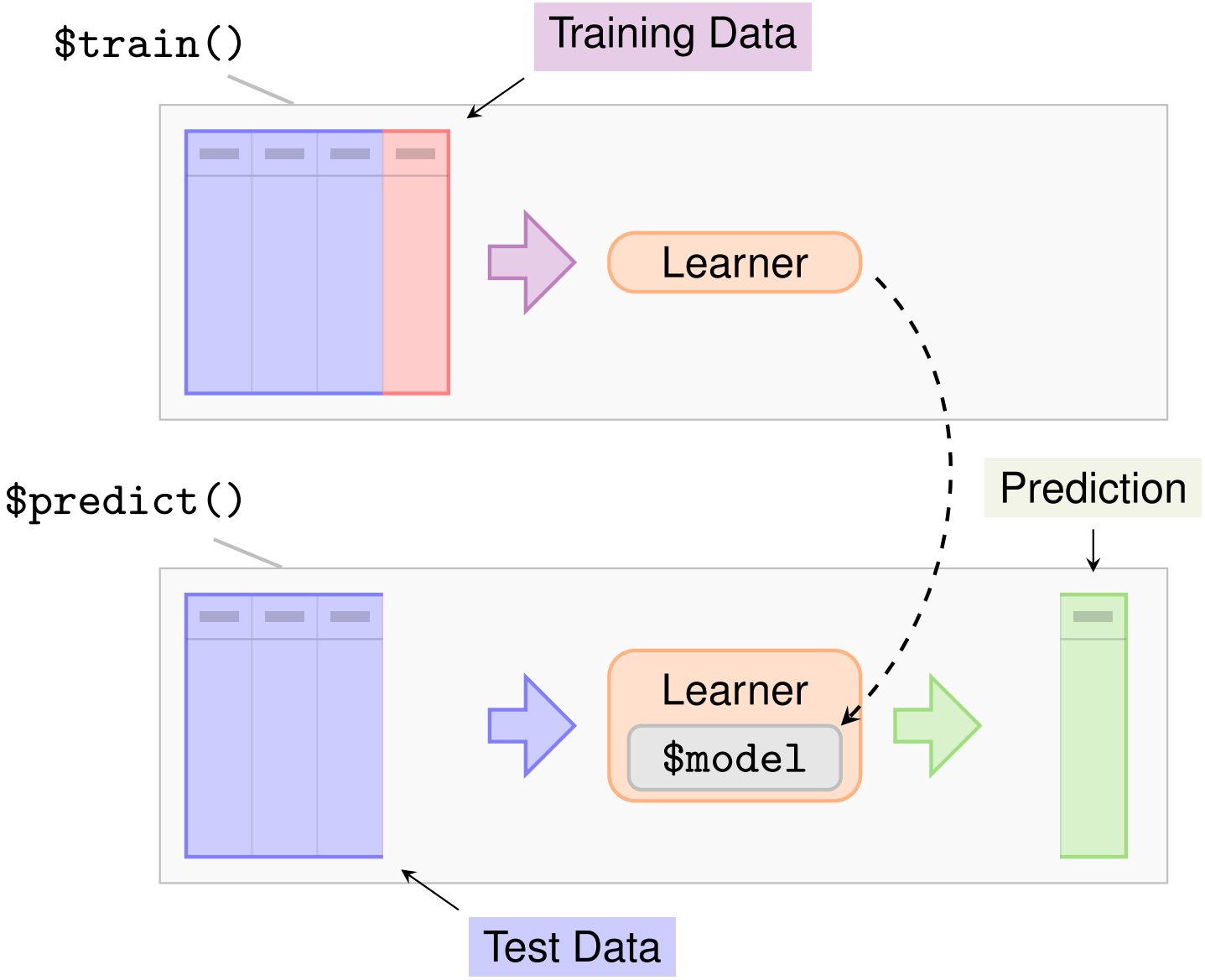
LEARNING ALGORITHMS



LEARNING ALGORITHMS



LEARNING ALGORITHMS



LEARNING ALGORITHMS

- Get a `Learner` provided by `mlr`

```
learner = lrn("classif.rpart")
```

LEARNING ALGORITHMS

- Get a `Learner` provided by `mlr`

```
learner = lrn("classif.rpart")
```

- Train the `Learner`

```
learner$train(task)
```

LEARNING ALGORITHMS

- Get a `Learner` provided by `mlr`

```
learner = lrn("classif.rpart")
```

- Train the `Learner`

```
learner$train(task)
```

- The `$model` is the `rpart` model: a decision tree

```
print(learner$model)

#> n= 150
#>
#> node), split, n, loss, yval, (yprob)
#>       * denotes terminal node
#>
#> 1) root 150 100 setosa (0.333 0.333 0.333)
#>   2) Petal.Length< 2.4 50   0 setosa (1.000 0.000 0.000) *
#>   3) Petal.Length>=2.4 100  50 versicolor (0.000 0.500 0.500)
#>     6) Petal.Width< 1.8 54   5 versicolor (0.000 0.907 0.093) *
#>     7) Petal.Width>=1.8 46   1 virginica (0.000 0.022 0.978) *
```

HYPERPARAMETERS

- Learners have *hyperparameters* that control their behavior

```r
as.data.table(learner$param_set)[, 1:6]

#>                id    class lower upper      levels nlevels
#>             <char>   <char> <num> <num>      <list>   <num>
#>  1:             cp ParamDbl     0     1                 Inf
#>  2:     keep_model ParamLgl    NA    NA  TRUE,FALSE       2
#>  3:     maxcompete ParamInt     0   Inf                 Inf
#>  4:       maxdepth ParamInt     1    30                  30
#>  5:   maxsurrogate ParamInt     0   Inf                 Inf
#>  6:      minbucket ParamInt     1   Inf                 Inf
#>  7:       minsplit ParamInt     1   Inf                 Inf
#>  8: surrogatestyle ParamInt     0     1                   2
#>  9:   usesurrogate ParamInt     0     2                   3
#> 10:           xval ParamInt     0   Inf                 Inf
```

HYPERPARAMETERS

- Changing hyperparameters after the creation of a `Learner` object:

```
learner$param_set$values = list(maxdepth = 1, xval = 0)
```

- Changing hyperparameters when the `Learner` object is created:

```
learner = lrn("classif.rpart", maxdepth = 1, xval = 0)
```

HYPERPARAMETERS

- Changing hyperparameters after the creation of a `Learner` object:

```
learner$param_set$values = list(maxdepth = 1, xval = 0)
```

- Changing hyperparameters when the `Learner` object is created:

```
learner = lrn("classif.rpart", maxdepth = 1, xval = 0)
```

- The `Learner` behavior changes and gives a smaller decision tree:

```
learner$train(task)
print(learner$model)
#> n= 150
#>
#> node), split, n, loss, yval, (yprob)
#>       * denotes terminal node
#>
#> 1) root 150 100 setosa (0.33 0.33 0.33)
#>   2) Petal.Length< 2.4 50   0 setosa (1.00 0.00 0.00) *
#>   3) Petal.Length>=2.4 100  50 versicolor (0.00 0.50 0.50) *
```

PREDICTION

- Let's make a prediction for some new data, e.g.:

```
new_data
#   Sepal.Length Sepal.Width Petal.Length Petal.Width
# 1            4           3            2           1
# 2            2           2            3           2
```

PREDICTION

- Let's make a prediction for some new data, e.g.:

```
new_data
#   Sepal.Length Sepal.Width Petal.Length Petal.Width
# 1            4           3            2           1
# 2            2           2            3           2
```

- To do so, we call the `$predict_newdata()` method using the new data:

```
prediction = learner$predict_newdata(new_data)
```

PREDICTION

- Let's make a prediction for some new data, e.g.:

```
new_data
#   Sepal.Length Sepal.Width Petal.Length Petal.Width
# 1            4           3            2           1
# 2            2           2            3           2
```

- To do so, we call the `$predict_newdata()` method using the new data:

```
prediction = learner$predict_newdata(new_data)
```

- We get a `Prediction` object:

```
prediction
#> <PredictionClassif> for 2 observations:
#>  row_ids truth   response
#>        1  <NA>     setosa
#>        2  <NA> versicolor
```

PREDICTION

- Let's make a prediction for some new data, e.g.:

```
new_data
#   Sepal.Length Sepal.Width Petal.Length Petal.Width
# 1            4           3            2           1
# 2            2           2            3           2
```

- To do so, we call the `$predict_newdata()` method using the new data:

```
prediction = learner$predict_newdata(new_data)
```

- We get a `Prediction` object:

```
prediction
#> <PredictionClassif> for 2 observations:
#>  row_ids truth   response
#>        1  <NA>     setosa
#>        2  <NA> versicolor
```

PREDICTION

- We can make the `Learner` predict *probabilities* when we set `predict_type`:

```
learner$predict_type = "prob"
learner$predict_newdata(new_data)
```

```
# <PredictionClassif> for 2 observations:
#  row_ids truth  response prob.setosa prob.versicolor
#        1  <NA>    setosa           1             0.0
#        2  <NA> virginica           0             0.5
#  prob.virginica
#             0.0
#             0.5
```

PREDICTION

What exactly is a `Prediction` object?

- Contains predictions and offers useful access fields / methods

PREDICTION

What exactly is a `Prediction` object?

- Contains predictions and offers useful access fields / methods

⇒ Use `as.data.table()` to extract data

```
as.data.table(prediction)
#>    row_ids  truth   response
#>      <int> <fctr>     <fctr>
#> 1:       1   <NA>     setosa
#> 2:       2   <NA> versicolor
```

PREDICTION

What exactly is a `Prediction` object?

- Contains predictions and offers useful access fields / methods

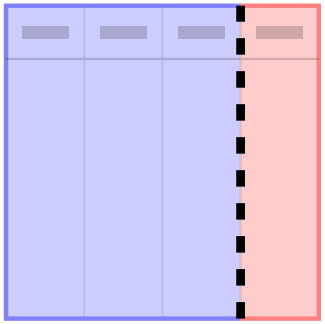
⇒ Use `as.data.table()` to extract data

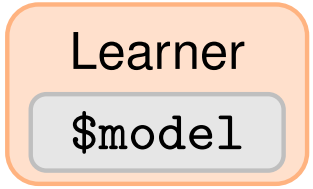
```
as.data.table(prediction)
#>    row_ids  truth   response
#>      <int> <fctr>     <fctr>
#> 1:       1   <NA>     setosa
#> 2:       2   <NA> versicolor
```

⇒ Active bindings and functions that give further information: `$response`, `$truth`, ...

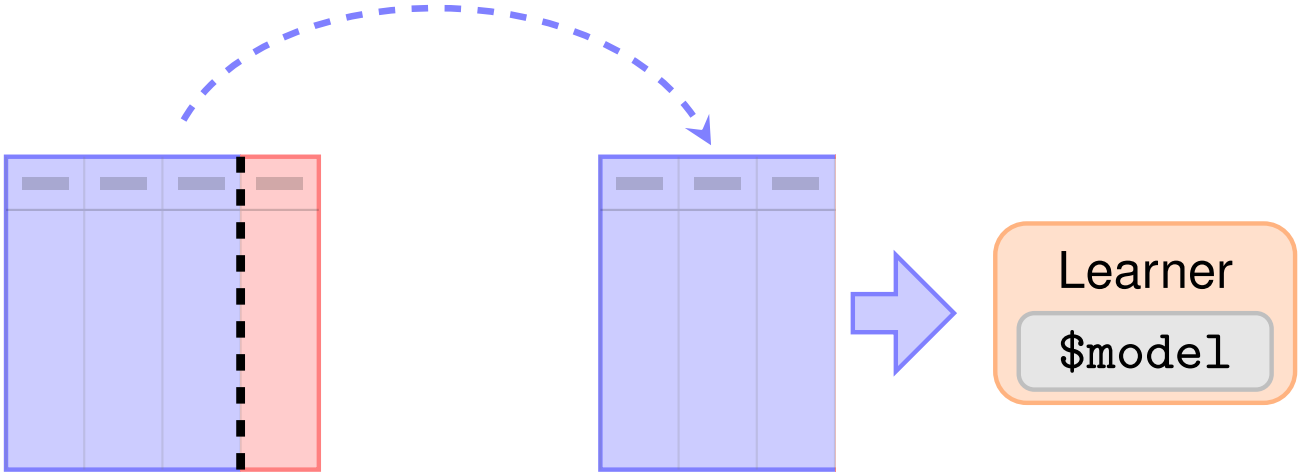
```
prediction$response
#> [1] setosa     versicolor
#> Levels: setosa versicolor virginica
```

PERFORMANCE EVALUATION

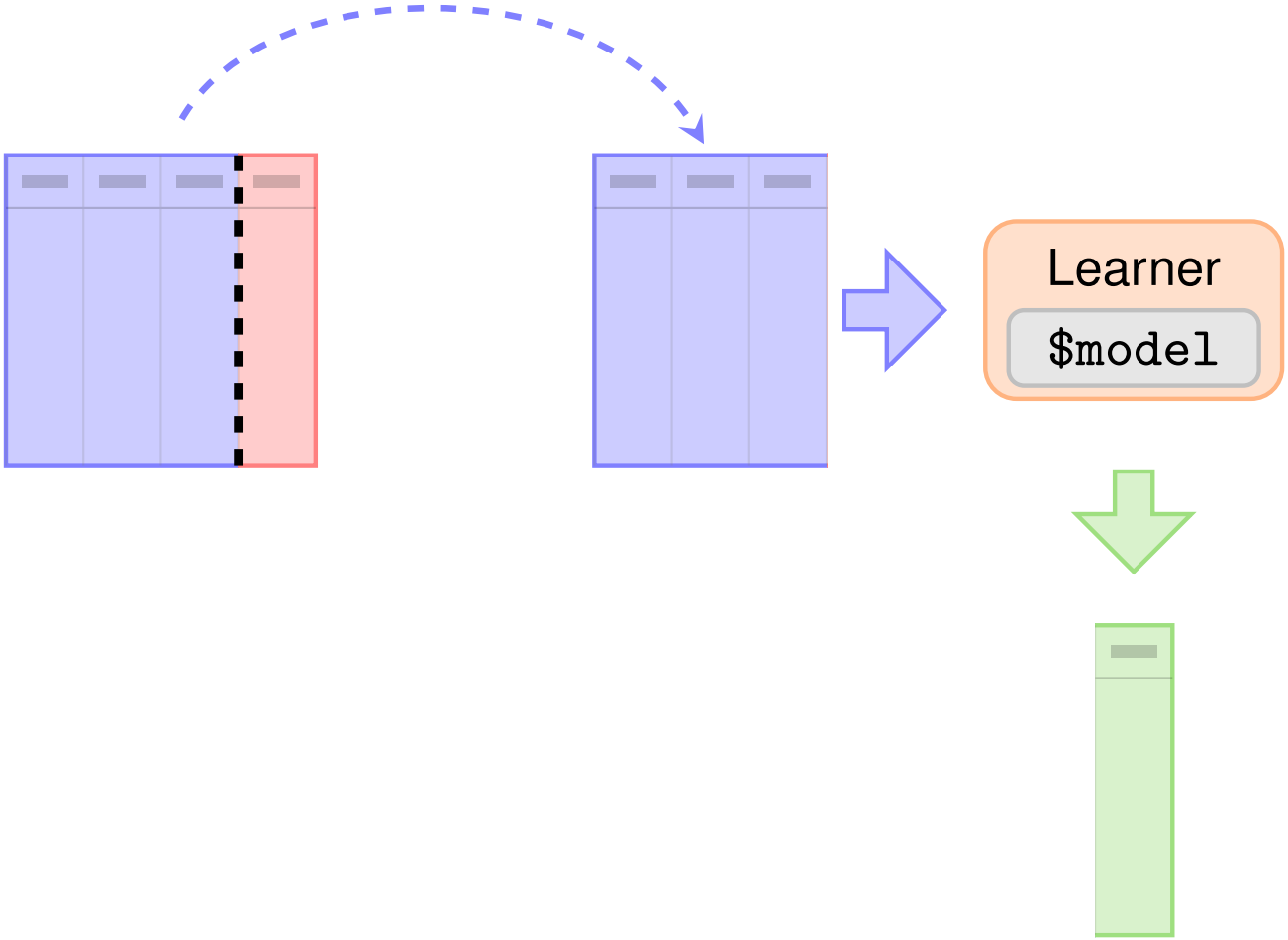




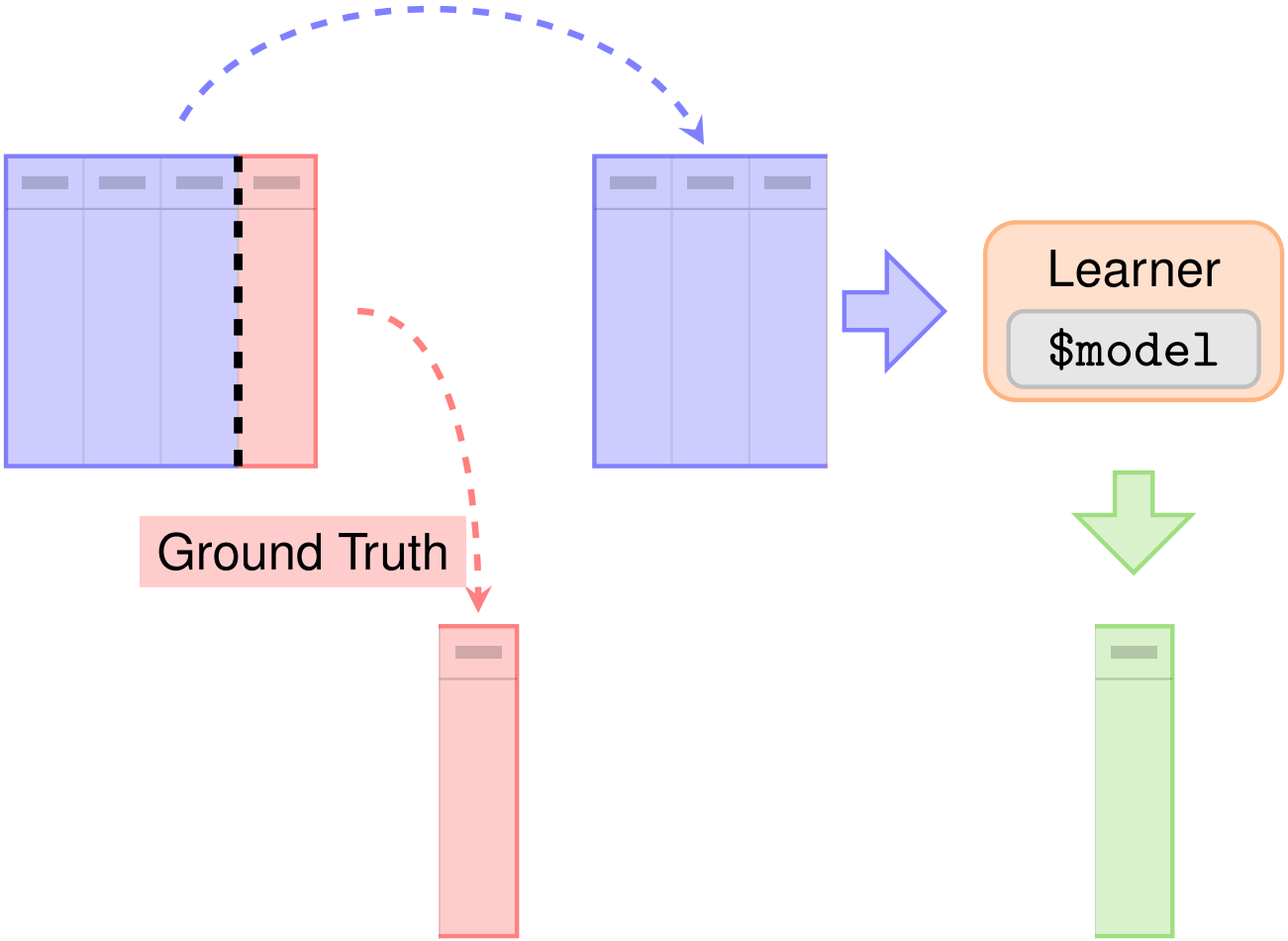
PERFORMANCE EVALUATION



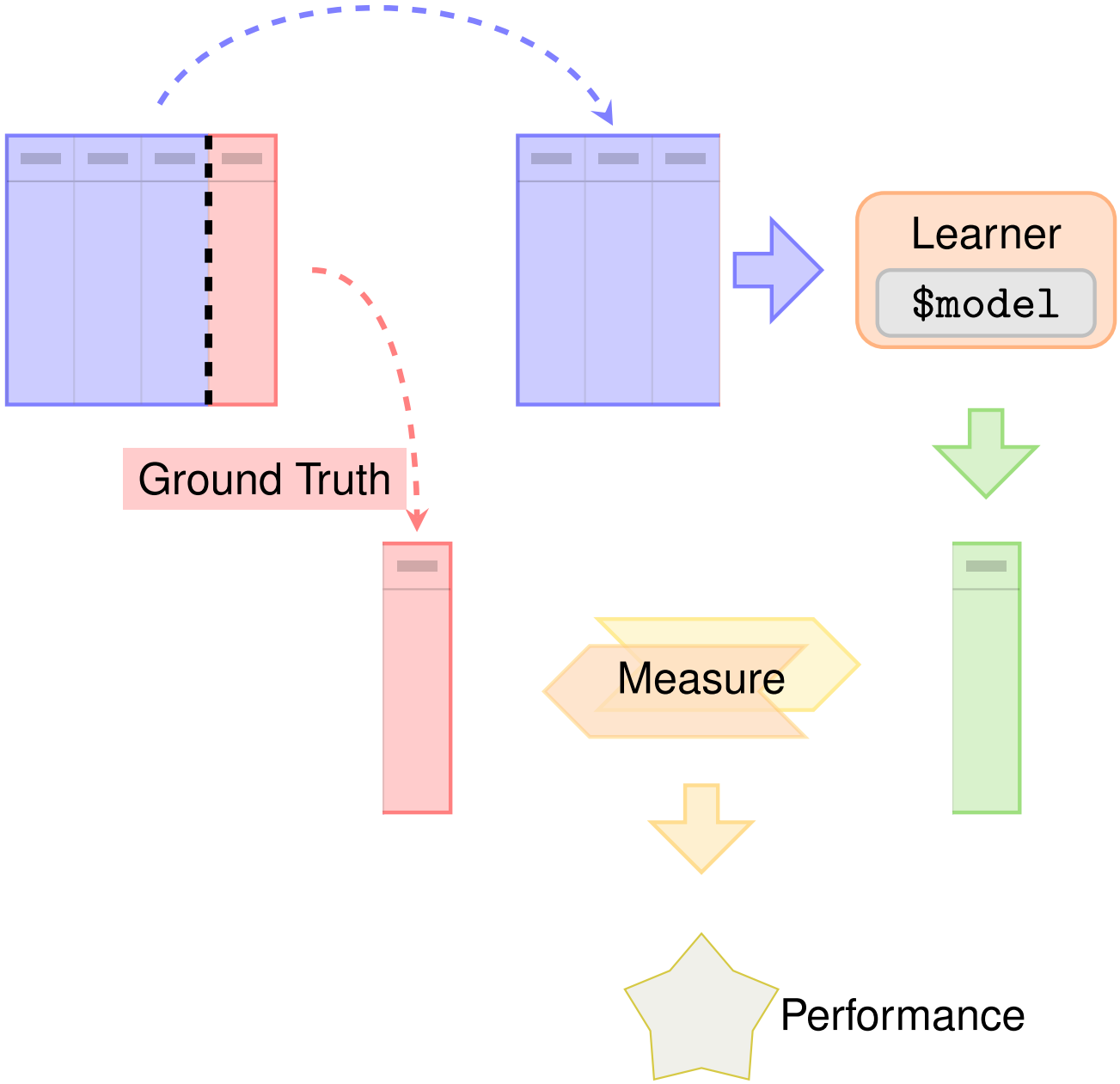
PERFORMANCE EVALUATION



PERFORMANCE EVALUATION



PERFORMANCE EVALUATION



PERFORMANCE EVALUATION

- Prediction ‘Task‘ with known data

```
known_truth_task$data()
#    Species Petal.Length Petal.Width Sepal.Length Sepal.Width
#     <fctr>        <num>       <num>        <num>       <num>
# 1:  setosa            2           1            4           3
# 2:  setosa            3           2            2           2
```

PERFORMANCE EVALUATION

- Prediction ‘Task‘ with known data

```
known_truth_task$data()
#    Species Petal.Length Petal.Width Sepal.Length Sepal.Width
#     <fctr>        <num>       <num>        <num>       <num>
# 1:  setosa            2           1            4           3
# 2:  setosa            3           2            2           2
```

- Predict again

```
pred = learner$predict(known_truth_task)
pred
#> <PredictionClassif> for 2 observations:
#>  row_ids  truth  response
#>        1 setosa    setosa
#>        2 setosa virginica
```

PERFORMANCE EVALUATION

- Prediction ‘Task‘ with known data

```
known_truth_task$data()
#    Species Petal.Length Petal.Width Sepal.Length Sepal.Width
#     <fctr>        <num>       <num>        <num>       <num>
# 1:  setosa            2           1            4           3
# 2:  setosa            3           2            2           2
```

- Predict again

```
pred = learner$predict(known_truth_task)
pred
#> <PredictionClassif> for 2 observations:
#>  row_ids  truth  response
#>        1 setosa    setosa
#>        2 setosa virginica
```

- Score the prediction

```
pred$score(msr("classif.ce"))
#> classif.ce
#>        0.5
```

PERFORMANCE EVALUATION

- Prediction ‘Task‘ with known data

```
known_truth_task$data()
#    Species Petal.Length Petal.Width Sepal.Length Sepal.Width
#     <fctr>        <num>       <num>        <num>       <num>
# 1:  setosa            2           1            4           3
# 2:  setosa            3           2            2           2
```

- Predict again

```
pred = learner$predict(known_truth_task)
pred
#> <PredictionClassif> for 2 observations:
#>  row_ids  truth  response
#>        1 setosa    setosa
#>        2 setosa virginica
```

- Score the prediction

```
pred$score(msr("classif.ce"))
#> classif.ce
#>        0.5
```

APPLIED MACHINE LEARNING

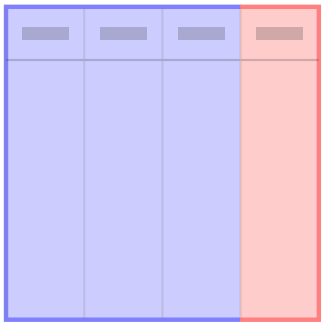
Introduction to mlr3
- Motivation and R6
- Data and Tasks
- Dictionaries
- Learner
- Prediction and Evaluation

Resampling with mlr3

Benchmarking with mlr3

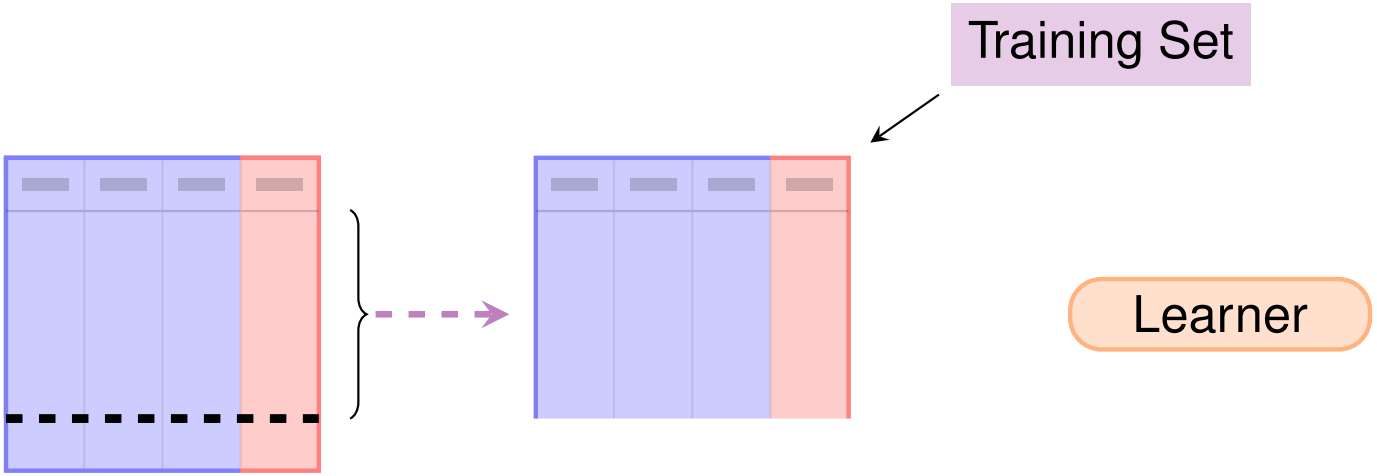
Help and Summary

RESAMPLING

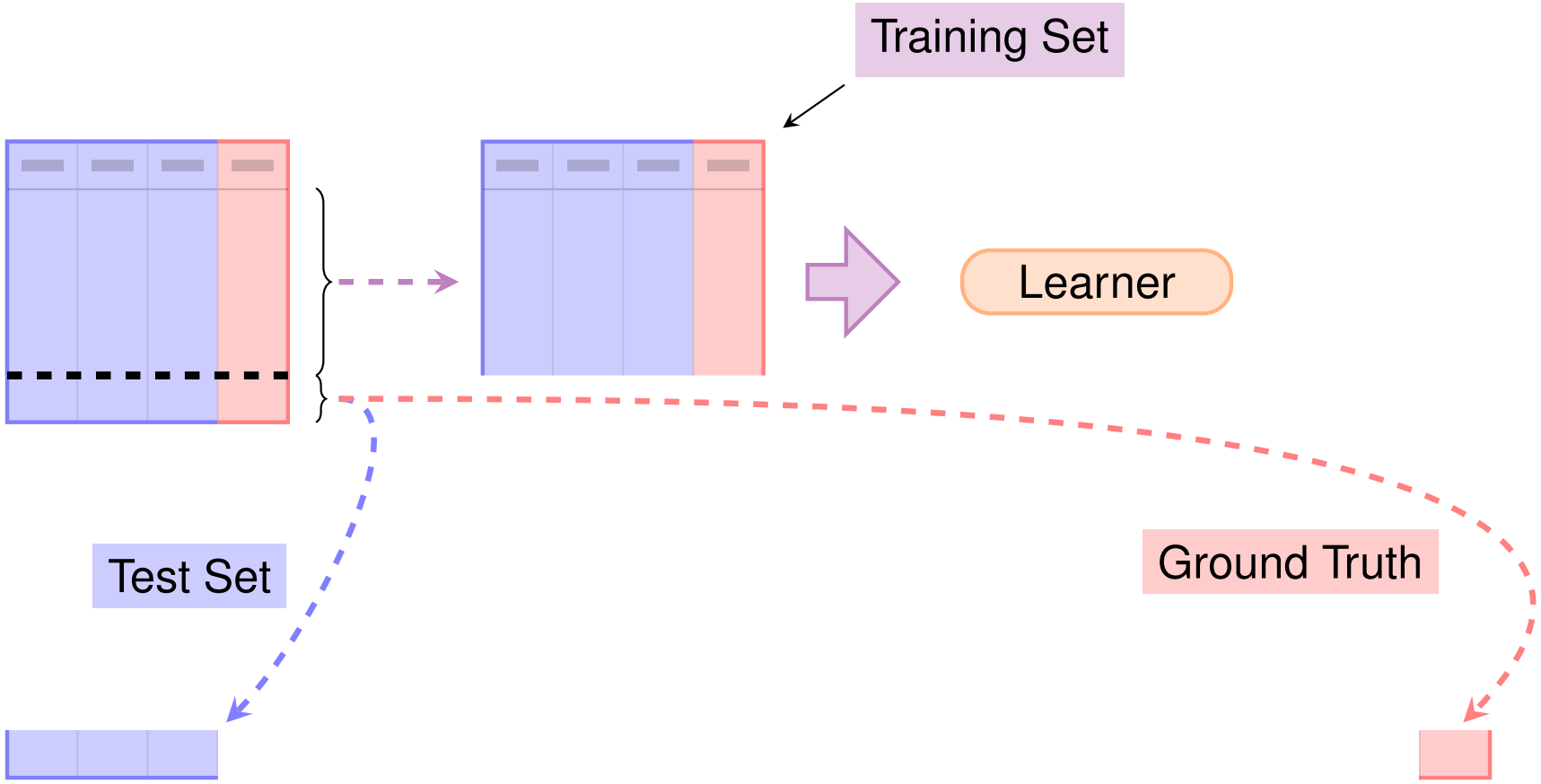


Learner

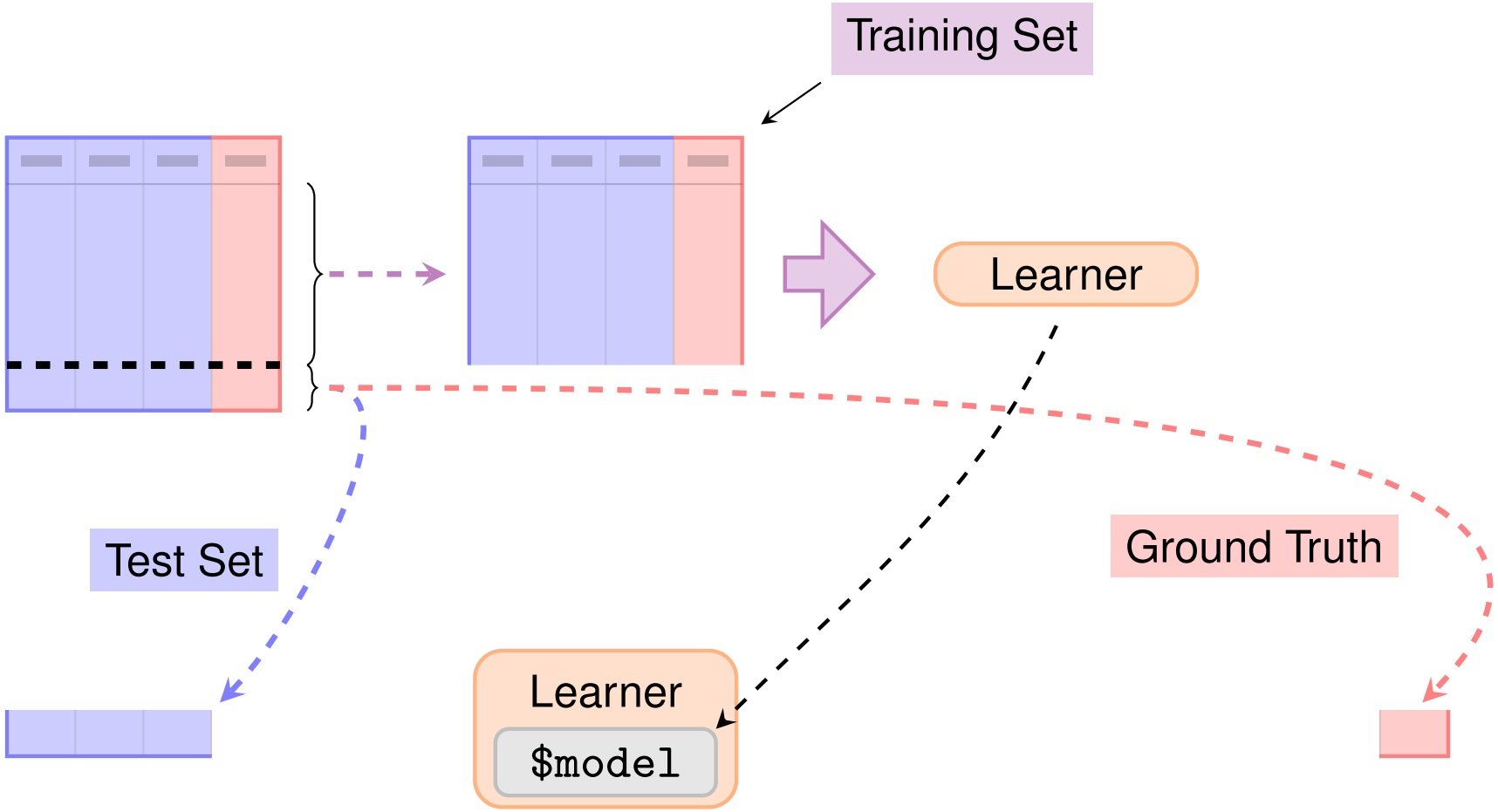
RESAMPLING



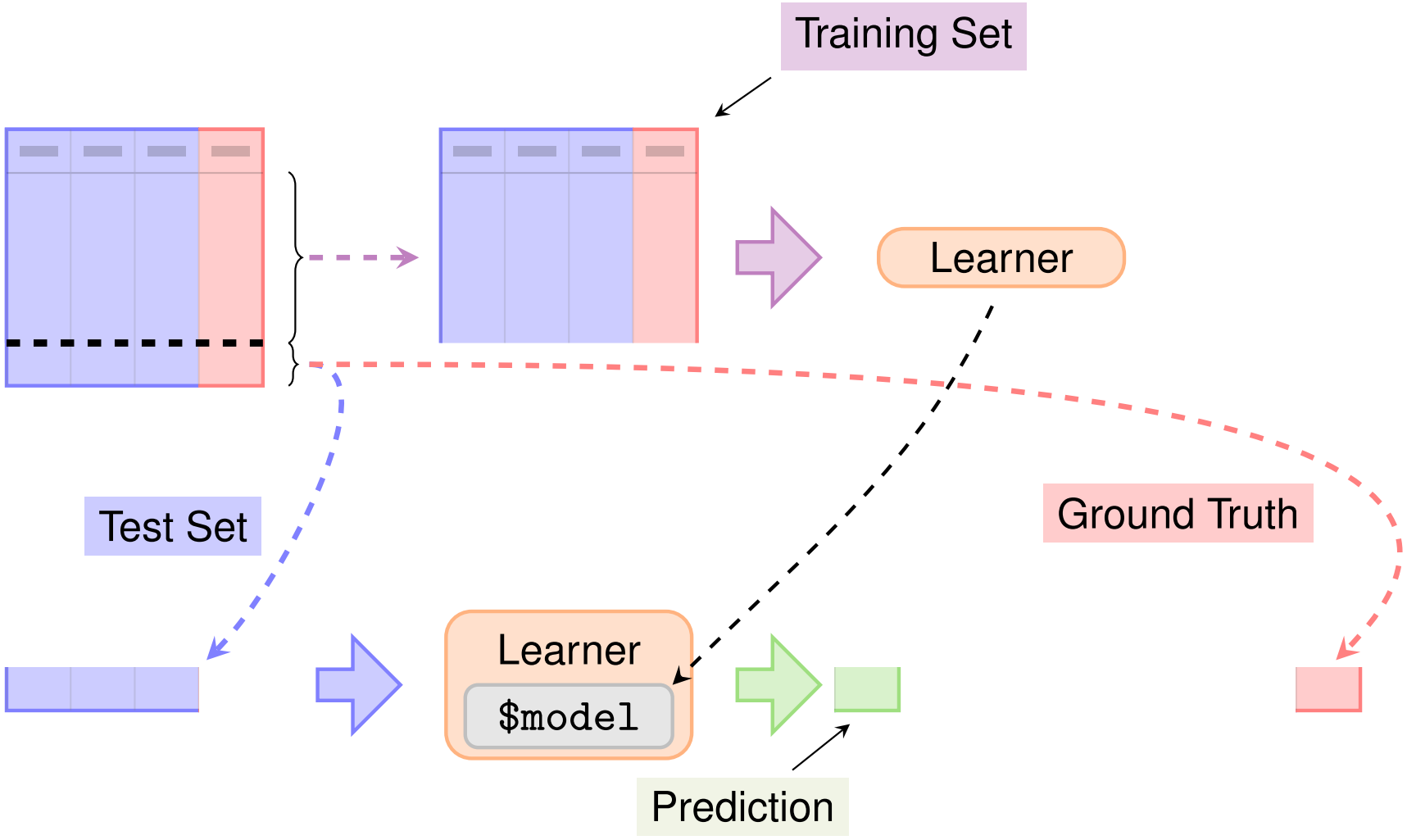
RESAMPLING



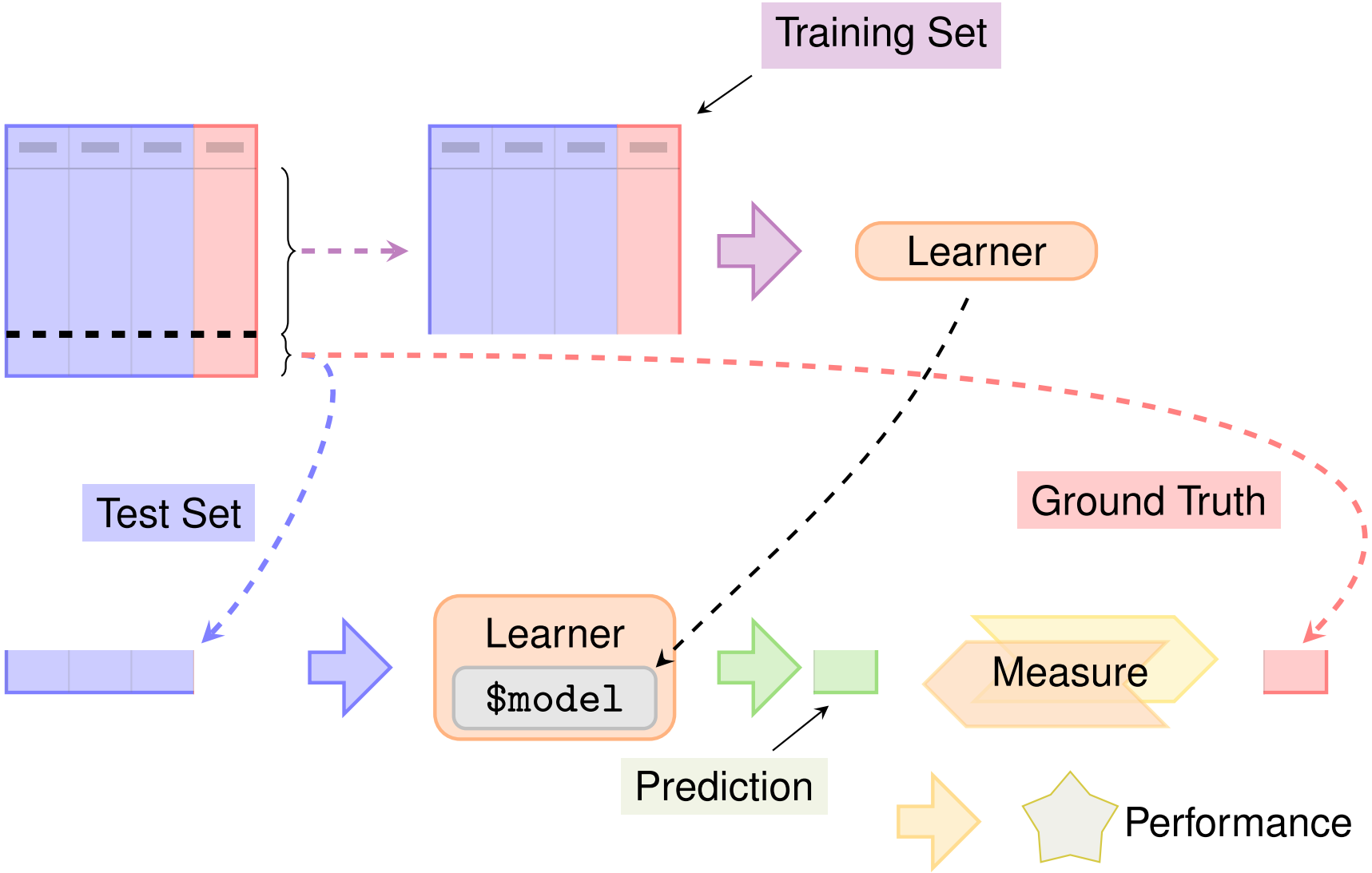
RESAMPLING



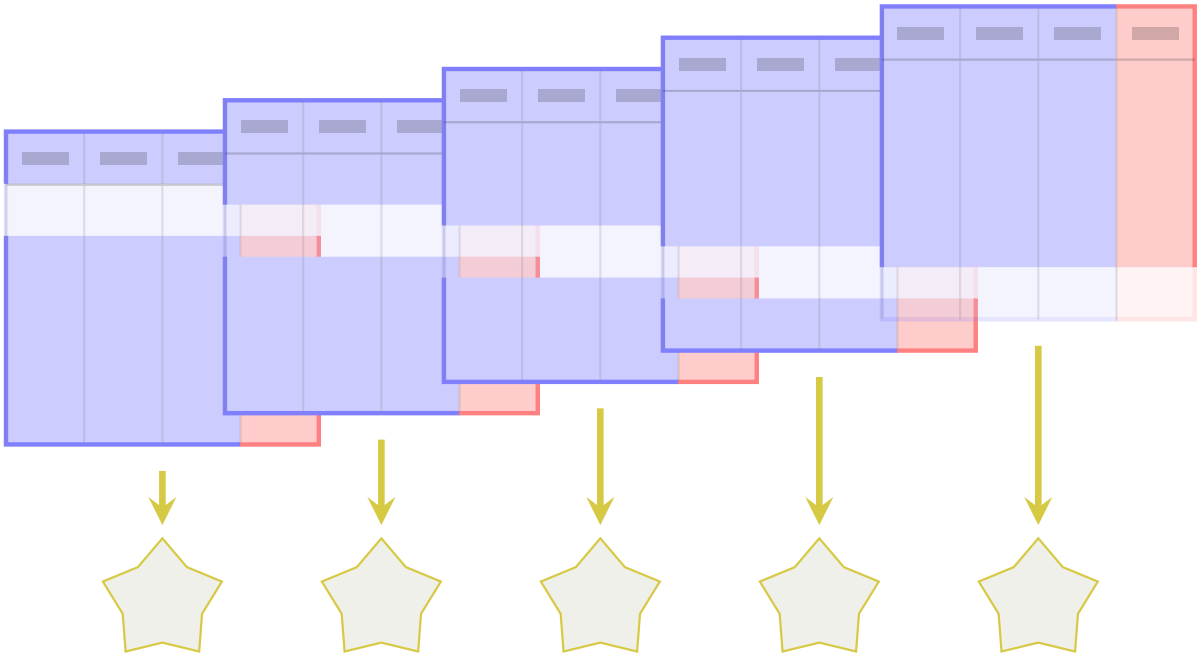
RESAMPLING


Training Set
Learner
Test Set
Ground Truth
Learner
$model
Prediction

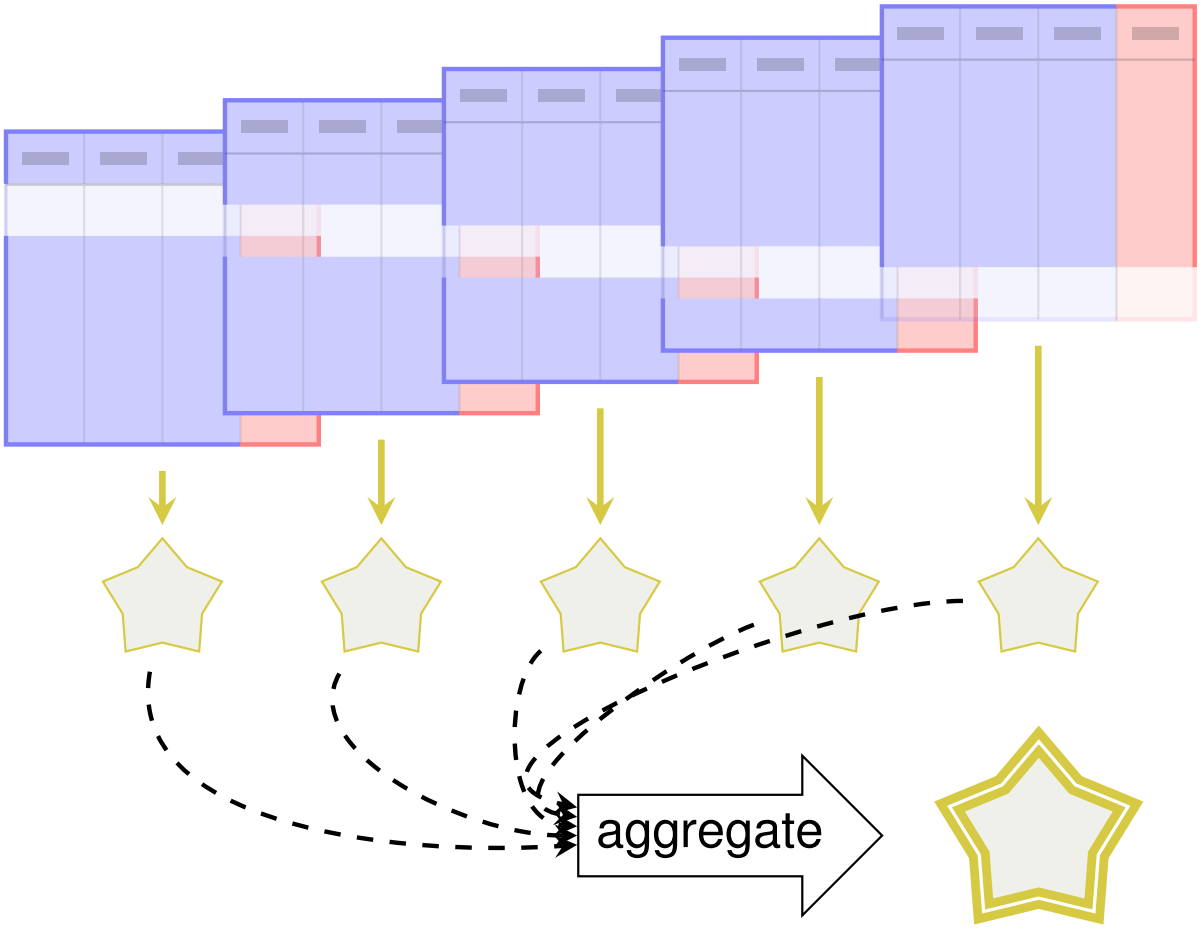
RESAMPLING


Training Set
Learner
Test Set
Ground Truth
Learner
$model
Measure
Prediction
Performance

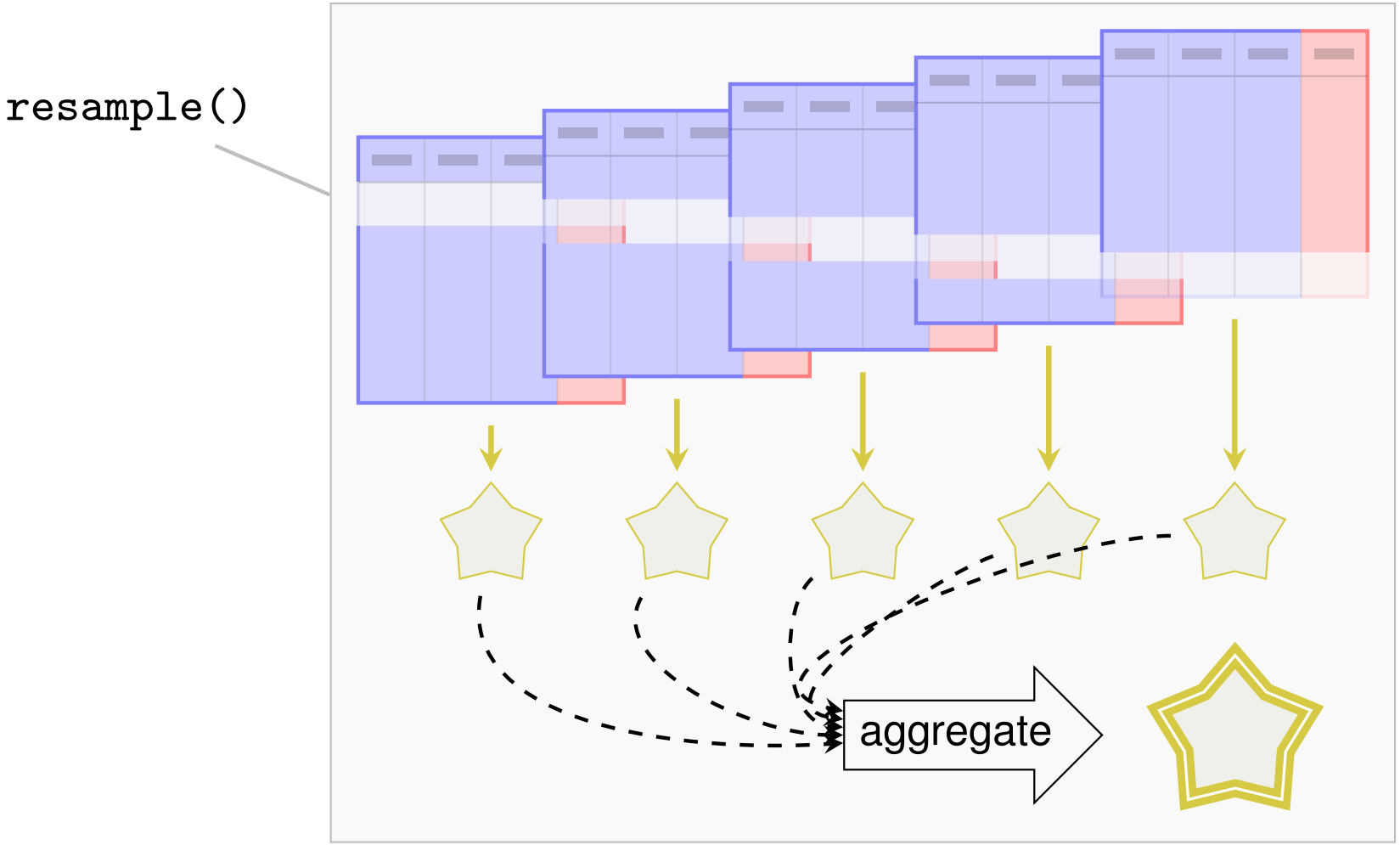
RESAMPLING



RESAMPLING



RESAMPLING



RESAMPLING

- Resample description: How to split the data

```
cv5 = rsmp("cv", folds = 5)
```

RESAMPLING

- Resample description: How to split the data

```
cv5 = rsmp("cv", folds = 5)
```

- Use the `resample()` function for resampling:

```
task = as_task_classif(x = iris, target = "Species", id = "iris")
learner = lrn("classif.rpart")
rr = resample(task, learner, cv5)
```

RESAMPLING

- Resample description: How to split the data

```
cv5 = rsmp("cv", folds = 5)
```

- Use the `resample()` function for resampling:

```
task = as_task_classif(x = iris, target = "Species", id = "iris")
learner = lrn("classif.rpart")
rr = resample(task, learner, cv5)
```

- We get a `ResamplingResult` object:

```
print(rr)
#> <ResampleResult> with 5 resampling iterations
#>  task_id    learner_id resampling_id iteration warnings errors
#>     iris classif.rpart            cv         1        0      0
#>     iris classif.rpart            cv         2        0      0
#>     iris classif.rpart            cv         3        0      0
#>     iris classif.rpart            cv         4        0      0
#>     iris classif.rpart            cv         5        0      0
```

RESAMPLING RESULTS

What exactly is a `ResamplingResult` object?
Remember `Prediction`:

- Get a table representation using `as.data.table()`

```
rr_table = as.data.table(rr)

print(rr_table)
#                   task                                learner
#                 <list>                                 <list>
# 1: <TaskClassif:iris> <LearnerClassifRpart:classif.rpart>
# 2: <TaskClassif:iris> <LearnerClassifRpart:classif.rpart>
# 3: <TaskClassif:iris> <LearnerClassifRpart:classif.rpart>
# 4: <TaskClassif:iris> <LearnerClassifRpart:classif.rpart>
# 5: <TaskClassif:iris> <LearnerClassifRpart:classif.rpart>
#        resampling iteration          prediction
#            <list>     <int>              <list>
# 1: <ResamplingCV>         1 <PredictionClassif>
# 2: <ResamplingCV>         2 <PredictionClassif>
# 3: <ResamplingCV>         3 <PredictionClassif>
# 4: <ResamplingCV>         4 <PredictionClassif>
# 5: <ResamplingCV>         5 <PredictionClassif>
```

- Active bindings and functions that make information easily accessible

RESAMPLING RESULTS

- Calculate performance:

```
rr$aggregate(msr("classif.ce"))
#> classif.ce
#>       0.06
```

RESAMPLING RESULTS

- Calculate performance:

```
rr$aggregate(msr("classif.ce"))
#> classif.ce
#>       0.06
```

- Get predictions

```
rr$prediction()
#> <PredictionClassif> for 150 observations:
#>     row_ids     truth  response
#>           2    setosa    setosa
#>           7    setosa    setosa
#>           9    setosa    setosa
#> ---
#>         141 virginica virginica
#>         148 virginica virginica
#>         149 virginica virginica
```

RESAMPLING

- Predictions of individual folds

```
predictions = rr$predictions()
predictions[[1]]
#> <PredictionClassif> for 30 observations:
#>     row_ids     truth   response
#>           2    setosa     setosa
#>           7    setosa     setosa
#>           9    setosa     setosa
#> ---
#>         135 virginica versicolor
#>         136 virginica  virginica
#>         142 virginica  virginica
```

RESAMPLING

- Predictions of individual folds

```
predictions = rr$predictions()
predictions[[1]]
#> <PredictionClassif> for 30 observations:
#>     row_ids     truth   response
#>           2    setosa     setosa
#>           7    setosa     setosa
#>           9    setosa     setosa
#> ---
#>         135 virginica versicolor
#>         136 virginica  virginica
#>         142 virginica  virginica
```

- Score of individual folds

```
scores = rr$score()
scores[1:3, c("iteration", "classif.ce")]
#>    iteration classif.ce
#>        <int>      <num>
#> 1:         1      0.067
#> 2:         2      0.100
#> 3:         3      0.067
```

APPLIED MACHINE LEARNING

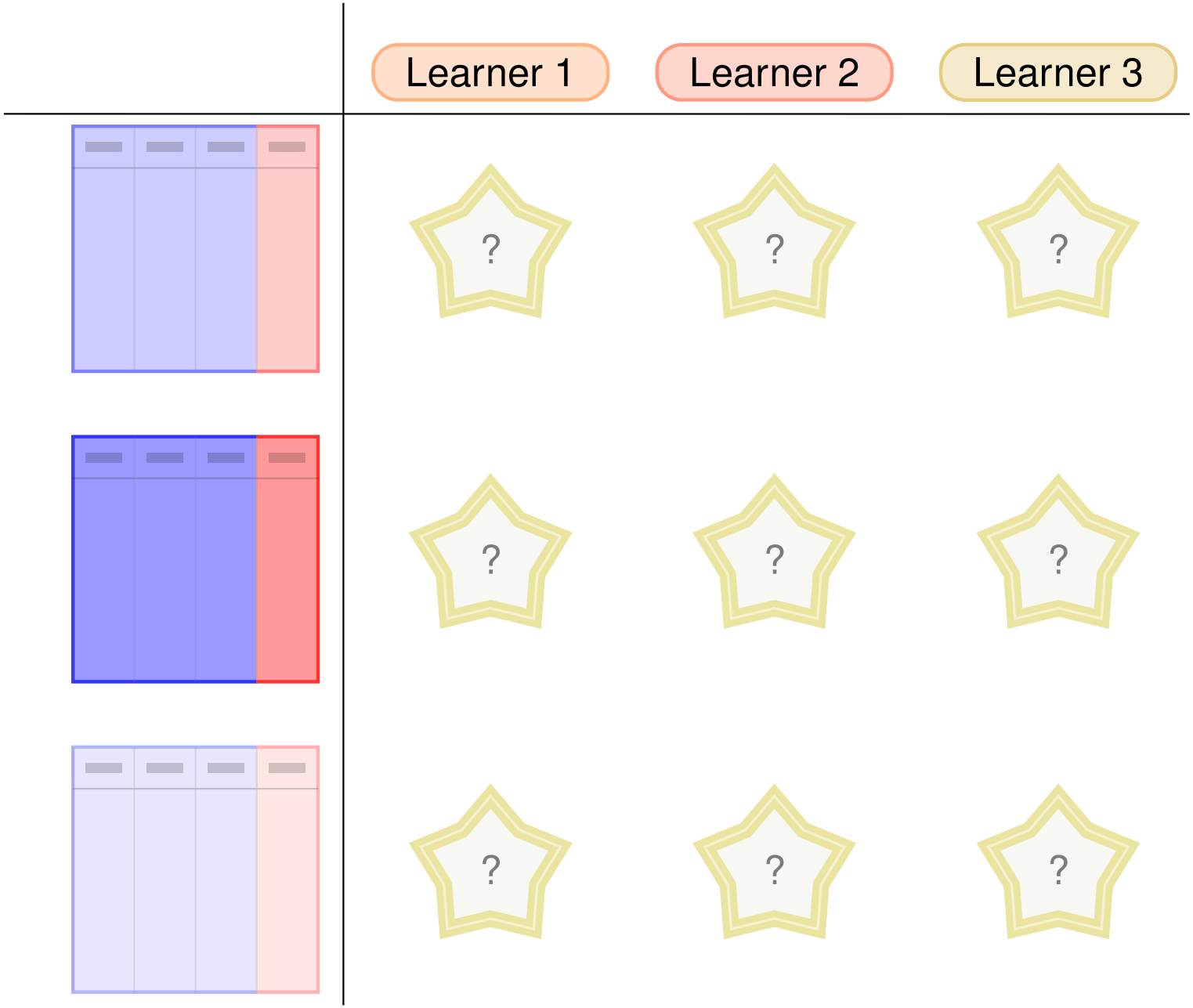
Introduction to mlr3
- Motivation and R6
- Data and Tasks
- Dictionaries
- Learner
- Prediction and Evaluation

Resampling with mlr3

Benchmarking with mlr3

Help and Summary

PERFORMANCE COMPARISON



PERFORMANCE COMPARISON

- Multiple Learners, multiple Tasks:

```
library("mlr3learners")
learners = list(lrn("classif.rpart"), lrn("classif.kknn"))
tasks = list(tsk("iris"), tsk("sonar"), tsk("wine"))
```

PERFORMANCE COMPARISON

- Multiple Learners, multiple Tasks:

```
library("mlr3learners")
learners = list(lrn("classif.rpart"), lrn("classif.kknn"))
tasks = list(tsk("iris"), tsk("sonar"), tsk("wine"))
```

- Set up the *design* and execute benchmark:

```
design = benchmark_grid(tasks, learners, cv5)
bmr = benchmark(design)
```

PERFORMANCE COMPARISON

- Multiple Learners, multiple Tasks:

```
library("mlr3learners")
learners = list(lrn("classif.rpart"), lrn("classif.kknn"))
tasks = list(tsk("iris"), tsk("sonar"), tsk("wine"))
```

- Set up the *design* and execute benchmark:

```
design = benchmark_grid(tasks, learners, cv5)
bmr = benchmark(design)
```

- We get a `BenchmarkResult` object which shows that `kknn` outperforms `rpart`:

```
bmr_ag = bmr$aggregate()
bmr_ag[, c("task_id", "learner_id", "classif.ce")]
#>    task_id    learner_id classif.ce
#>     <char>        <char>      <num>
#> 1:    iris classif.rpart      0.053
#> 2:    iris  classif.kknn      0.040
#> 3:   sonar classif.rpart      0.274
#> 4:   sonar  classif.kknn      0.130
#> 5:    wine classif.rpart      0.157
#> 6:    wine  classif.kknn      0.039
```

BENCHMARK RESULT

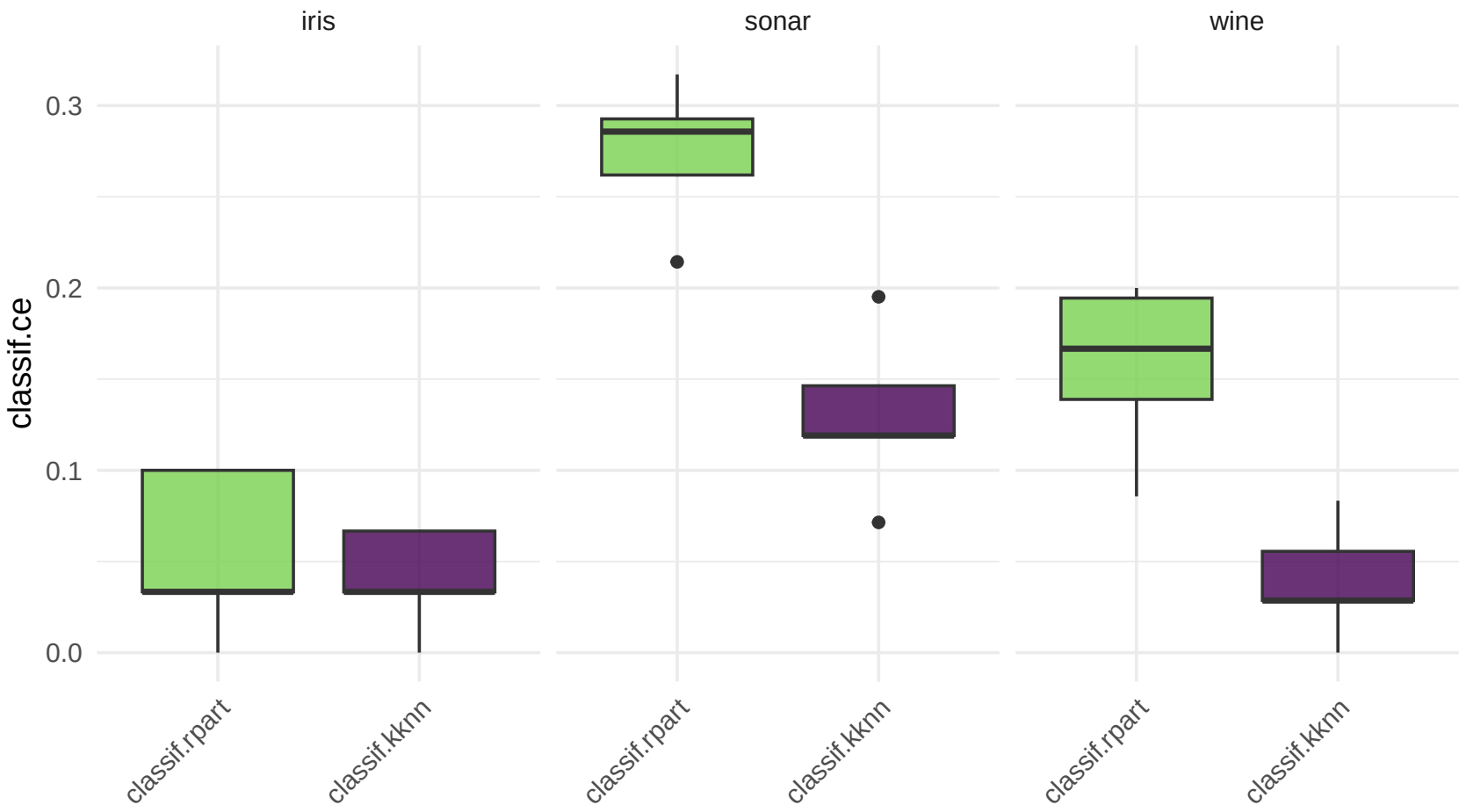
What exactly is a `BenchmarkResult` object?
Just like `Prediction` and `ResamplingResult`!

- Table representation using `as.data.table()`
- Active bindings and functions that make information easily accessible

BENCHMARK RESULT

The `mlr3viz` package contains `autoplot()` functions for many mlr3 objects

```
library(mlr3viz)
autoplot(bmr)
```



CONTROL OF EXECUTION

Parallelization

```
future::plan("multicore")
```

- runs each resampling iteration as a job
- also allows nested resampling (although not needed here)

Encapsulation

```
learner$encapsulate = c(train = "callr", predict = "callr")
```

- Spawns a separate R process to train the learner
- Learner may segfault without tearing down the session
- Logs are captured
- Possibilty to have a fallback to create predictions

APPLIED MACHINE LEARNING

Introduction to mlr3
- Motivation and R6
- Data and Tasks
- Dictionaries
- Learner
- Prediction and Evaluation

Resampling with mlr3

Benchmarking with mlr3

Help and Summary

HOW TO GET HELP

- Where to start?
  - Check these slides
  - **Check the mlr3book https://mlr3book.mlr-org.com**

HOW TO GET HELP

- Where to start?
  - Check these slides
  - **Check the mlr3book https://mlr3book.mlr-org.com**
- Get help for R6 objects?
  1. Find out what kind of R6 object you have:

     ```
     class(bmr)
     #> [1] "BenchmarkResult" "R6"
     ```

  2. Go to the corresponding help page:

     ```
     ?BenchmarkResult
     ```

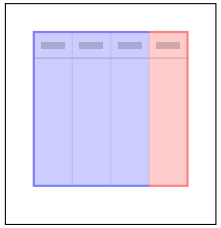
  New: open the corresponding man page with

  ```
  learner$help()
  ```

OVERVIEW

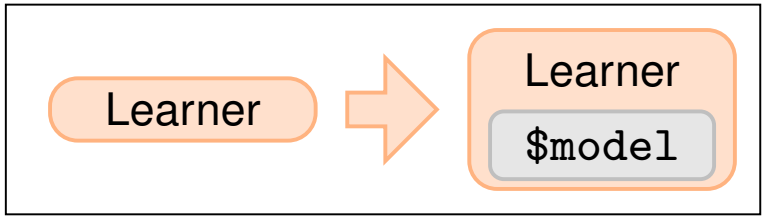
Ingredients:

Data



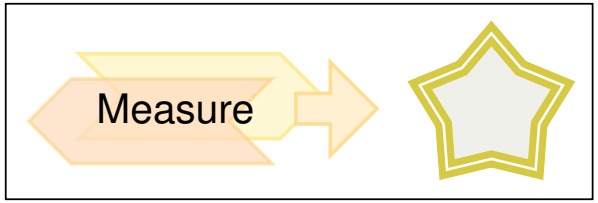
`TaskClassif`,
`TaskRegr`,
`tsk()`

Learning Algorithms



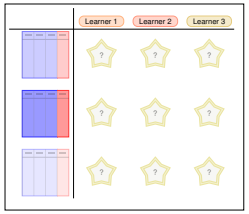
`lrn()` ⇒ `Learner`,
↪`Learner$train()`,
↪`Learner$predict()` ⇒ `Prediction`

Performance Evaluation



`rsmp()` ⇒ `Resampling`,
`msr()` ⇒ `Measure`,
`resample()` ⇒ `ResamplingResult`,
↪ `ResamplingResult$score()`,
↪ `ResamplingResult$aggregate()`

Performance Comparison



`benchmark_grid()`,
`benchmark()` ⇒ `BenchmarkResult`